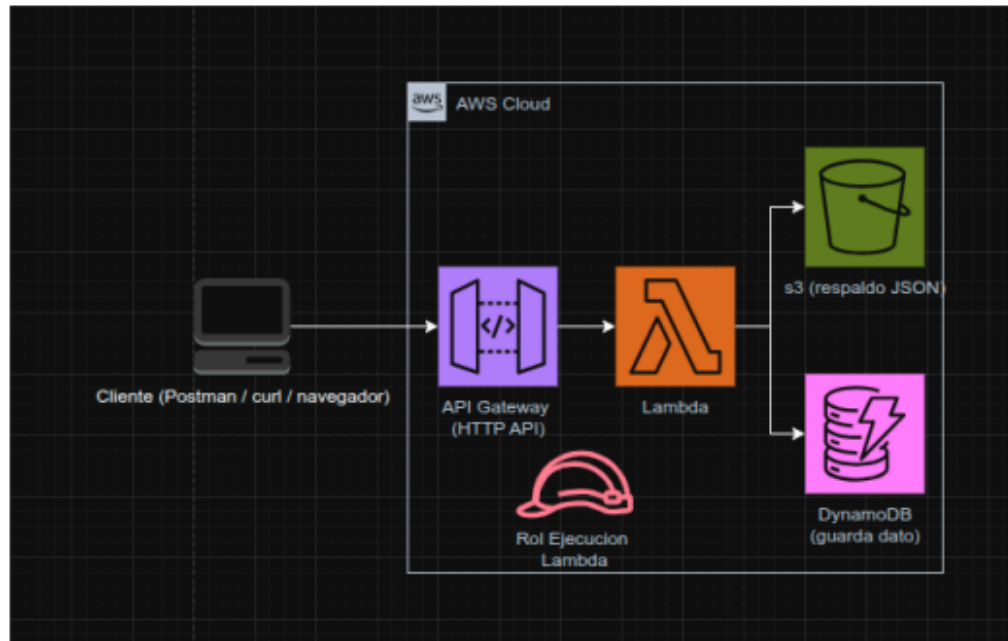


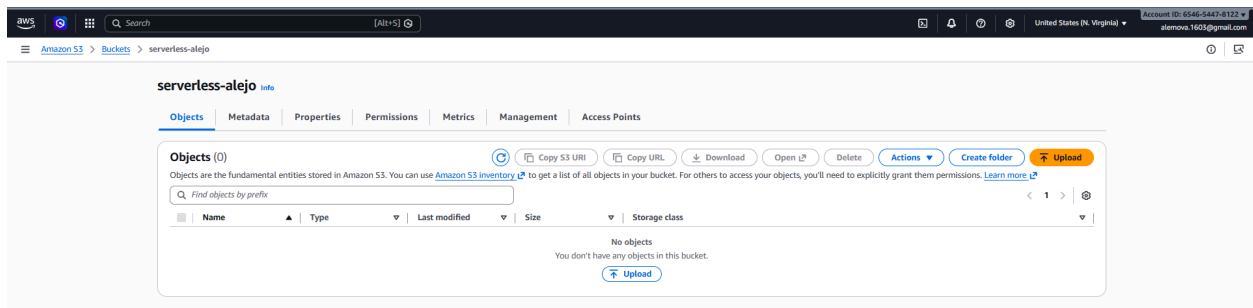
Evidencia clase serverless - 28/03/2026



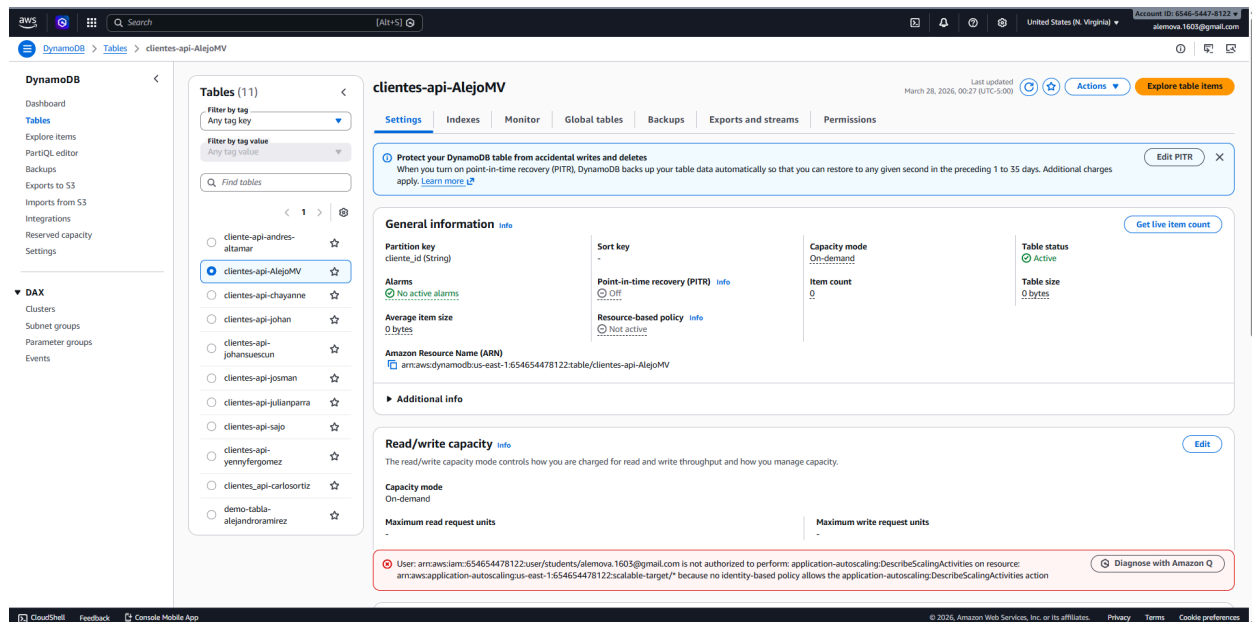
Arquitectura aplicada



1. Creación S3



2. Creación tabla DynamoDB



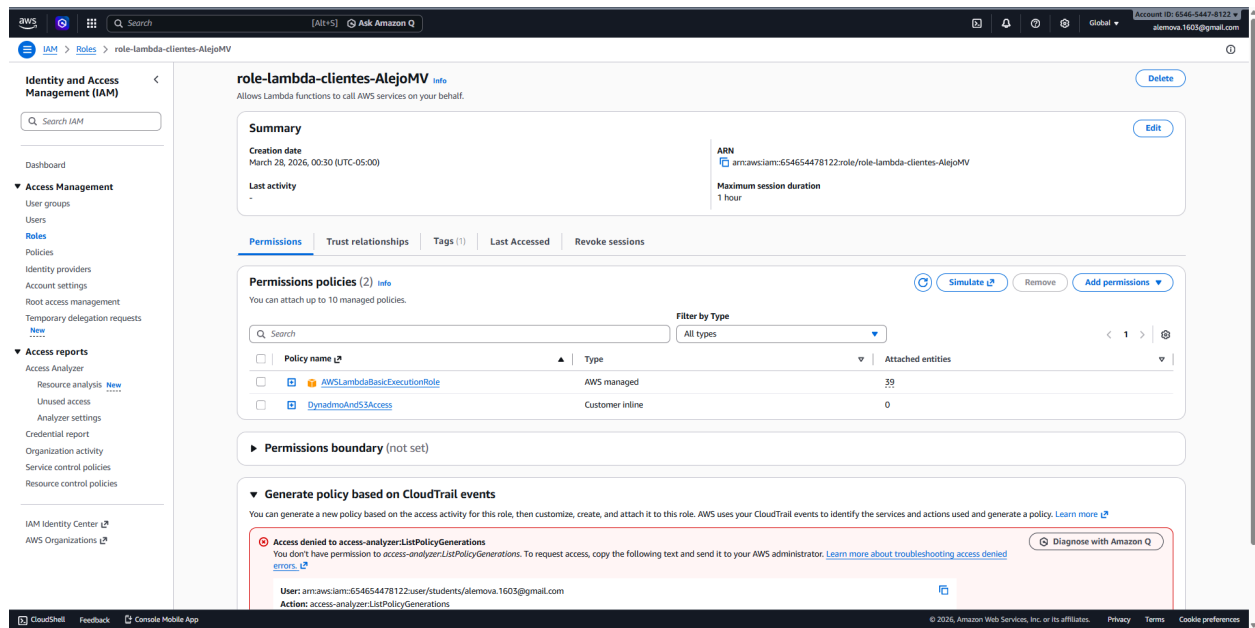
3. Creación de rol para la lambda

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "dynamodb:PutItem",
        "dynamodb:GetItem"
      ],
      "Resource": "arn:aws:dynamodb:us-east-1:654654478122:table/clientes-api-AlejoMV"
    },
    {
      "Effect": "Allow",
```

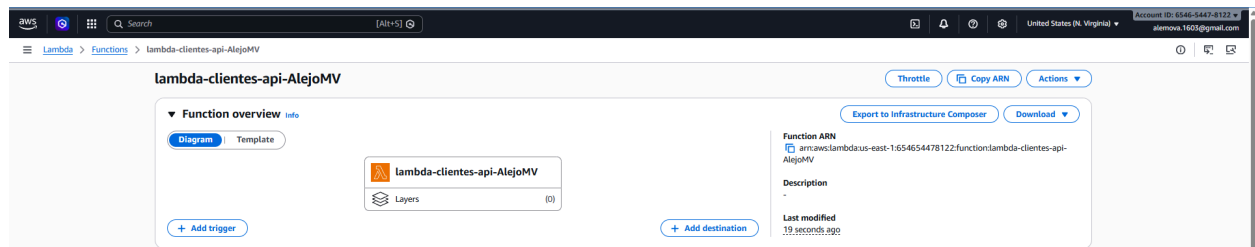
```

    "Action": [
      "s3:PutObject"
    ],
    "Resource": "arn:aws:s3:::serverless-alejo/*"
  }
]
}

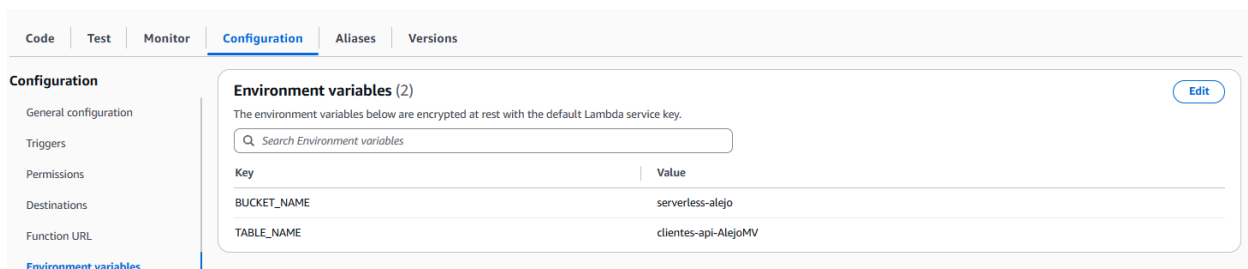
```



4. Creación función lambda



Configuramos variables de entorno



Agregamos el código

```
import json
import os
import uuid
from datetime import datetime
import boto3

dynamodb = boto3.resource('dynamodb')
s3 = boto3.client('s3')

def response(status_code, body):
    return {
        "statusCode": status_code,
        "headers": {
            "Content-Type": "application/json"
        },
        "body": json.dumps(body, default=str)
    }

def lambda_handler(event, context):
    print("Evento recibido:", json.dumps(event))

    try:
        TABLE_NAME = os.environ['TABLE_NAME']
        BUCKET_NAME = os.environ['BUCKET_NAME']
```

```

print("TABLE_NAME:", TABLE_NAME)
print("BUCKET_NAME:", BUCKET_NAME)

table = dynamodb.Table(TABLE_NAME)

method = event.get("requestContext", {}).get("http",
{}).get("method")
raw_path = event.get("rawPath", "")
path_params = event.get("pathParameters") or {}

# POST /clientes
if method == "POST" and raw_path == "/clientes":
    body = json.loads(event.get("body", "{}"))

    nombre = body.get("nombre")
    correo = body.get("correo")
    ciudad = body.get("ciudad")

    if not nombre or not correo or not ciudad:
        return response(400, {
            "message": "Los campos nombre, correo y c
iudad son obligatorios"
        })

    cliente_id = str(uuid.uuid4())
    fecha_registro = datetime.utcnow().isoformat()

    item = {
        "cliente_id": cliente_id,
        "nombre": nombre,
        "correo": correo,
        "ciudad": ciudad,
        "fecha_registro": fecha_registro
    }

    print("Guardando en DynamoDB...")

```

```

        table.put_item(Item=item)

    print("Guardando en S3...")
    s3.put_object(
        Bucket=BUCKET_NAME,
        Key=f"clientes/{cliente_id}.json",
        Body=json.dumps(item),
        ContentType="application/json"
    )

    return response(201, {
        "message": "Cliente creado exitosamente",
        "cliente_id": cliente_id,
        "s3_backup": f"clientes/{cliente_id}.json"
    })

# GET /clientes/{id}
if method == "GET" and raw_path.startswith("/cliente
s/"):
    cliente_id = path_params.get("id")

    if not cliente_id:
        return response(400, {"message": "ID no propo
rcionado"})

    result = table.get_item(Key={"cliente_id": client
e_id})
    item = result.get("Item")

    if not item:
        return response(404, {"message": "Cliente no
encontrado"})

    return response(200, {
        "message": "Cliente encontrado",
        "data": item

```

```

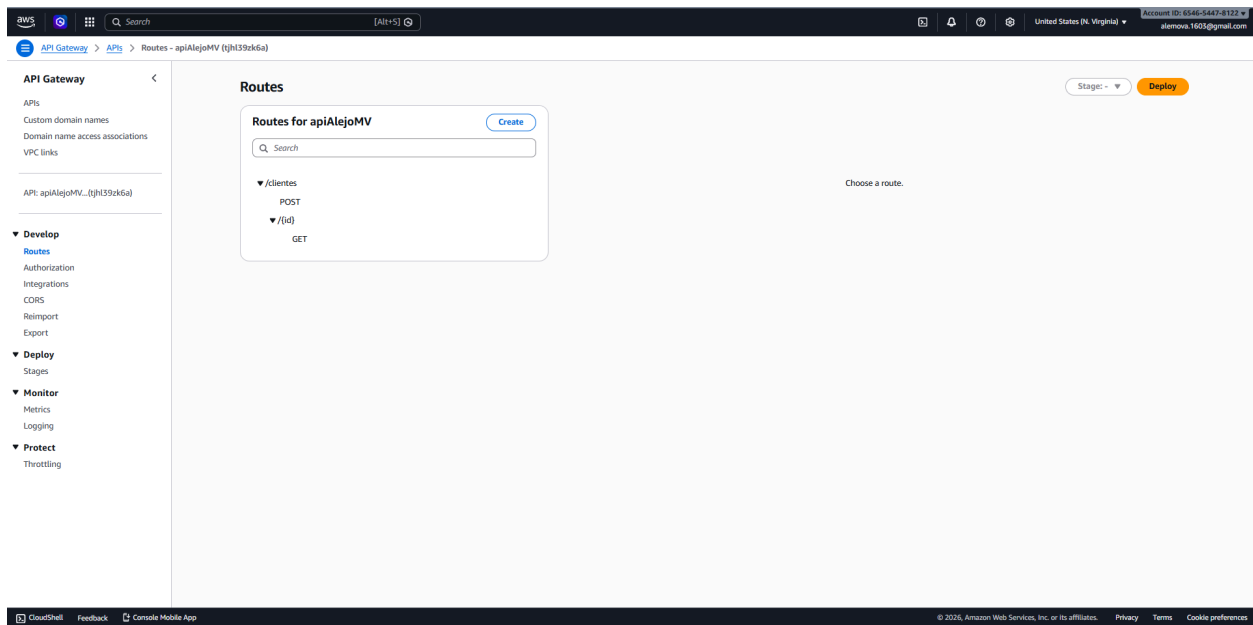
    })

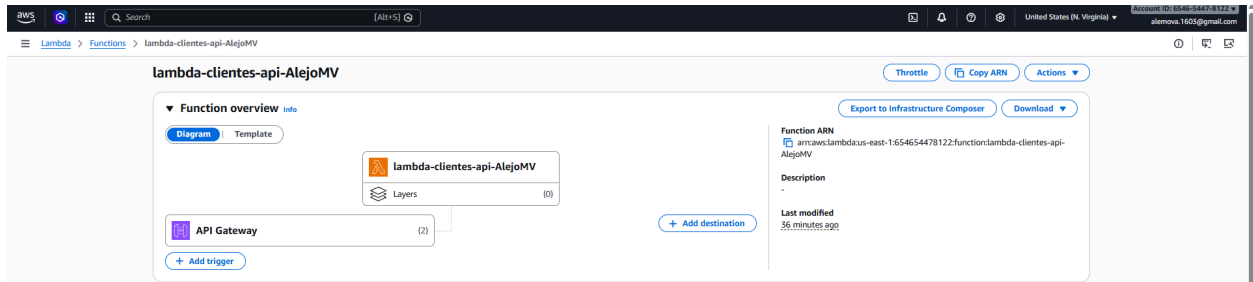
    return response(404, {"message": "Ruta no encontrada"})

except Exception as e:
    print("ERROR DETALLADO:", str(e))
    return response(500, {
        "message": "Error interno del servidor",
        "error": str(e)
    })

```

5. Creamos API HTTP en API Gateway





<https://tjhl39zk6a.execute-api.us-east-1.amazonaws.com>

6. Pruebas funcionales

1. Creación de un usuario con POST (user)

```
@baseUrl = https://tjhl39zk6a.execute-api.us-east-1.amazonaws.com
```

```
### Crear cliente
```

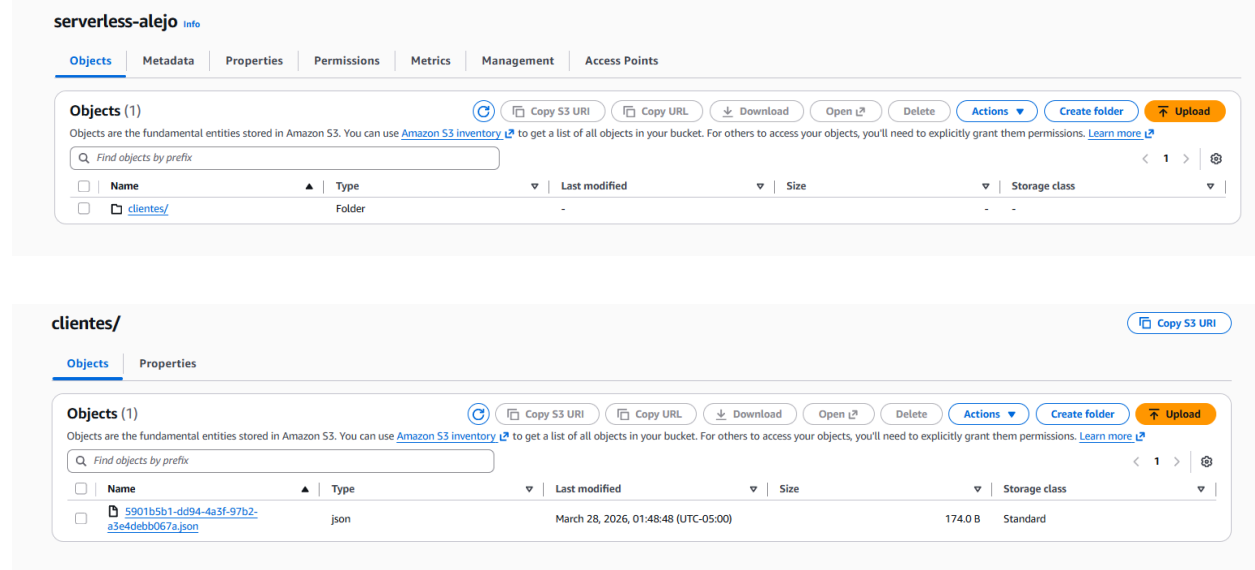
```
POST {{baseUrl}}/clientes
```

```
Content-Type: application/json
```

```
{  
  "nombre": "Johan",  
  "correo": "johan@demo.com",  
  "ciudad": "Bogotá"  
}
```


The screenshot shows the REST Client extension in VS Code. The left pane displays the request configuration for a POST request to `/clientes` with a JSON body: `{ "nombre": "Johan", "correo": "johan@demo.com", "ciudad": "Bogotá" }`. The right pane shows the response, which is a 201 status code with a JSON body: `{ "message": "Cliente creado exitosamente", "cliente_id": "5901b5b1-dd94-4a3f-97b2-a3e4debb067a", "s3_backup": "clientes/5901b5b1-dd94-4a3f-97b2-a3e4debb067a.json" }`.

Vemos que en efecto lo creo



2. Creación de un usuario con CURL(console)

```

alejandro@DESKTOP-PHUCDGV:~$ curl -X POST "https://tjhl39zk6a.execute-api.us-east-1.amazonaws.com/clientes" \
-H "Content-Type: application/json" \
-d '{
  "nombre": "Johan",
  "correo": "johan@democurl.com",
  "ciudad": "Bogotá"
}'
{"message": "Cliente creado exitosamente", "cliente_id": "d0b4a543-92fb-4cc9-8f3b-fc62de3112b3", "s3_backup": "clientes/d0b4a543-92fb-4cc9-8f3b-fc62de3112b3.json"}
alejandro@DESKTOP-PHUCDGV:~$

```

3. Recuperación de un usuario con GET

Obtener cliente por ID

```

GET https://tjhl39zk6a.execute-api.us-east-1.amazonaws.com/clientes/5901b5b1-dd94-4a3f-97b2-a3e4debb067a
Content-Type: application/json

```

The screenshot shows the REST Client extension in VS Code. The left pane displays a REST client script with a GET request to retrieve a client by ID. The right pane shows the response, which is a JSON object containing the client's details.

```

1 reference
2 @baseUrl = https://tjhl39zk6a.execute-api.us-east-1.amazonaws.com
3
4 ## Crear cliente
5 POST {{baseUrl}}/clientes
6 Content-Type: application/json
7 {
8   "nombre": "Johan",
9   "correo": "johan@demo.com",
10  "ciudad": "Bogotá"
11 }
12
13 ## Obtener cliente por ID
14 GET https://tjhl39zk6a.execute-api.us-east-1.amazonaws.com/clientes/5901b5b1-dd94-4a3f-97b2-a3e4debb067a
15 Content-Type: application/json

```

```

1 HTTP/1.1 200 OK
2 Date: Sat, 28 Mar 2026 06:58:02 GMT
3 Content-Type: application/json
4 Content-Length: 217
5 Connection: close
6 Apiga-Requestid: a78PPHNDaMEYde-
7
8 {
9   "message": "Cliente encontrado",
10  "data": {
11    "nombre": "Johan",
12    "correo": "johan@demo.com",
13    "fecha_registro": "2026-03-28T06:48:46.712525",
14    "cliente_id": "5901b5b1-dd94-4a3f-97b2-a3e4debb067a",
15    "ciudad": "Bogot\u00e1"
16  }
17 }

```

```

alejandro@DESKTOP-PHUCDGV:~$ curl -X GET "https://tjhl39zk6a.execute-api.us-east-1.amazonaws.com/clientes/5901b5b1-dd94-4a3f-97b2-a3e4debb067a"
{"message": "Cliente encontrado", "data": {"nombre": "Johan", "correo": "johan@demo.com", "fecha_registro": "2026-03-28T06:48:46.712525", "cliente_id": "5901b5b1-dd94-4a3f-97b2-a3e4debb067a", "ciudad": "Bogot\u00e1"}}
alejandro@DESKTOP-PHUCDGV:~$

```